Technical report — Los emal!

- **Track:** Track 2B — Los emal! (understand Zurich German with Apertus)
- **Event:** Online
- **Team:** Alina Ponomareva
- **Demo:** `link to video`

1. Summary

Newcomers to Zurich typically learn Standard German, yet everyday life (shops, offices, neighbours) happens in Zurich German. Answering in Standard German is socially accepted; the actual barrier is *understanding* the dialect. Los emal! is a listening-and-speaking trainer built around this asymmetry. In ten everyday scenes, Apertus 1.5 70B plays a local and speaks Zurich German, which is voiced by an open text-to-speech model; the learner listens with the text hidden, answers in Standard German by voice, and receives a comprehension check, an explanation of the hardest dialect words, and feedback on their Standard German.

The design follows from four evaluations. Apertus' experimental audio input does not understand spoken Zurich German well enough for speech input (WER 88.8 % against 30.9 % for Whisper on 100 SwissDial clips, E1). Apertus generates passable Zurich German but mixes in Bernese and Basel forms; a short Zurich style guide in the prompt raises the share of Zurich forms from 61 % to 70 % without improving chrF (E3). Crucially, it cannot judge whether a sentence is Zurich German: both models score below a judge that always says "no" (E2), so the app never grades the learner's dialect and restricts corrective feedback to Standard German. Even there, Apertus passed almost every learner error as correct while the check was part of the role-play prompt; as a separate, focused call it finds 13 of 20 errors without a false alarm on 43 correct sentences, and a separate comprehension check notices 18 of 20 misunderstandings instead of 3 (E4).

2. Architecture

See `docs/architecture.md` for the diagram. Per learner turn:

1. **Speech recognition** — Whisper large-v3-turbo (faster-whisper, int8 on CPU, CUDA if available; MLX on Apple Silicon) transcribes the learner's answer.
2. **Tutor** — `src/losemal/tutor.py` makes four Apertus calls per turn, at most two at a time. The *role-play call* sends the scenario prompt, the bounded conversation history and the transcript; the model returns one JSON object: the next line in Zurich German, Standard German and help-language translations, 1–3 hard dialect words and two suggested answers. Meanwhile three short calls run one after the other. The *Standard German check* asks only whether the learner's sentence is correct and how to fix it; the *comprehension check* asks only whether the answer fits what the local said; inside the role-play call the model passed almost every learner error as correct and missed most misunderstandings (E4). The *Zurich version call* renders the learner's sentence the way a local would say it, with the translation prompt and style guide evaluated in E3: inside the role-play call the model often returned its own line instead. Outputs are parsed tolerantly (code fences, trailing commas) and validated; the role-play output is repaired once if needed. Identical "corrections" and "wrong" verdicts without a correction are discarded, so the learner is never told to change a correct sentence; a check without a clear verdict shows no feedback rather than a "correct" the model never gave; suggestions in dialect or repeating what the learner already said are dropped.
3. **Speech synthesis** — Qwen3-TTS 12Hz Base (0.6B on CPU, 1.7B on GPU/Apple Silicon) speaks the Zurich German line by cloning a *synthetic* reference voice that we generated once

with Qwen3-TTS VoiceDesign from a text description (female and male voice). No real person's voice is cloned. Audio is cached; "slower" uses time-stretching.

4. **UI** — Gradio: listen first (text hidden), replay, slower, reveal text, translate, hints, word list per scene, interface and explanations in English, German or Russian, levels A2/B1/B2.

Scenario prompt. Each scene gives the model its role, what the character knows (documents for the registration, prices, free appointments, typical phrases) and a third-person description of the learner. An earlier version passed the learner's task card («Finde heraus, welche Dokumente du brauchst») verbatim; as «du» means the model everywhere else in the prompt, the clerk asked the learner which documents *she* needed (6 of 8 runs in a reproduction with 8B). The third-person description alone reduced but did not remove this; the character's knowledge and the rule «answer the learner's question first» were needed as well.

Robustness. During our runs, some requests to Public AI were rejected with HTTP 400 and succeeded on retry. We traced this to one of its backends, Featherless, which rejects every system message and every assistant turn (it accepts a single user turn only). The client retries with back-off; after such a rejection it folds the system prompt into the first user message and, if needed, sends the whole conversation as one user turn, and keeps that format for the rest of the session. The app therefore also works with Featherless directly (e.g. through the Hugging Face router), where it starts with the single-turn format.

Target architecture (mandatory)

The project is deployable **on-premise** and **air-gapped** (b), and in a **sovereign Swiss cloud** (c):

- Apertus is accessed through any OpenAI-compatible endpoint (`LLM_BASE_URL`): a self-hosted vLLM server with the official Swiss AI image (on-premise / air-gapped), or Swiss-hosted APIs (Public AI under Swiss jurisdiction, Infomaniak with data centres in Switzerland).
- Speech recognition and synthesis are open-weights models running inside the container.
- The learner's audio never leaves the container; only text is sent to the Apertus endpoint.

External dependencies. Build time: Python packages (PyPI, PyTorch CPU index) and the open weights from Hugging Face (Whisper large-v3-turbo, Qwen3-TTS 0.6B Base), baked into the image (6.6 GB). Runtime: only the Apertus endpoint, which can be local.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-70B` (default, via `LLM_NAME`); the 8B model works with the same prompts and was evaluated alongside it.
- **How it is used:** inference — role-play dialogue generation, Zurich German generation and translation in one structured JSON call per turn, and three short calls in parallel to it: feedback on the learner's Standard German, a comprehension check and the learner's sentence in Zurich German.
- **Where it runs:** hosted endpoint during development (Public AI, Swiss jurisdiction); any OpenAI-compatible endpoint, including self-hosted vLLM, at deployment.
- **Prompting:** a German system prompt (`src/losemal/prompts.py`) with the scenario, the learner's level, the help language, a compact *Zurich style guide* (Zurich vs. Bernese/Basel forms, common words, perfect tense only, formal address) and the JSON schema. Temperature 0.4 for the role-play, 0 for the short calls; thinking mode off (latency).
- **Other models and their role** (as required by the track rules): Whisper large-v3-turbo (speech recognition, MIT), Qwen3-TTS 12Hz Base and VoiceDesign (speech synthesis and creation of

the synthetic reference voices, Apache-2.0). Neither produces any learning content; all dialogue, explanations and feedback come from Apertus.

4. Data

- **SwissDial** (Dogan-Schönberger et al., 2021): parallel Swiss German / Standard German sentences with recordings for eight dialects, provided by ETH Zurich on request for research use. Used only for evaluation (E1–E3); it is **not** redistributed, and no SwissDial audio or voice is used in the app or the demo.
- **Synthetic learner test set** (`data/learner_de_testset.jsonl`, 63 items, written by the authors, CDLA-Permissive-2.0): Zurich German context lines from the app's scenarios with learner answers in Standard German — correct answers, answers with exactly one typical learner error, correct Swiss Standard German (*parkieren, grillieren, das Tram*) that must not be "corrected", and answers that are correct German but do not fit what was said (misunderstandings). No personal data.
- **Synthetic reference voices** (`data/voices/`), generated with Qwen3-TTS VoiceDesign.

5. Evaluation

All scripts are in `src/eval/` (`make eval-asr`, `eval-judge`, `eval-generation`, `eval-feedback`). 8B ran on Public AI. Public AI throttled the 70B model during our evaluation, so all 70B numbers in E2–E4 and the 70B latencies were obtained through the Hugging Face router (provider Featherless); that backend accepts a single user turn only, so instructions and history were sent as one user message (section 2).

E1 — Who understands spoken Zurich German? 100 randomly sampled SwissDial Zurich clips. Both systems were asked for the Standard German meaning of the clip (Zurich German speech → Standard German text), scored against the SwissDial Standard German sentence after normalisation. Since a correct paraphrase is also penalised, both metrics are rough; the gap between the systems is what matters.

System	WER ↓	chrF ↑	s / clip (Apple Silicon)
Whisper large-v3-turbo (MLX)	30.9 %	78.6	0.2
Apertus 1.5 8B, audio input	88.8 %	45.7	1.7

In 10 of the 100 clips Apertus did not attempt the task: in 8 it replied that it could not hear or receive audio («Es tut mir leid, aber ich kann keine Audiodateien hören …») although the audio tokens were part of the input, and in 2 it repeated the instruction. Excluding these 10 clips, Apertus reaches WER 78.0 % / chrF 48.7 (Whisper on the same 90 clips: 31.1 % / 79.0). It usually catches single words and numbers but invents the rest of the sentence, often fluently and with an unrelated meaning. We did not test whether a different prompt reduces the refusals. A 10-clip pilot had been noticeably more optimistic (Whisper 22.5 % WER). While running Apertus on Apple MPS we observed that `F.pad` with very large padding silently corrupted the output in PyTorch 2.14.1 on MPS; Apertus uses it to pad its 131,072 text logits to the full 266,752-token vocabulary, so every generated token was noise. Replacing it with an equivalent `torch.cat` made MPS and CPU outputs identical (same text, all 261 audio codes of a test clip), so the weak result is not an artefact of the platform. Decision: Whisper for speech input.

E2 — Can Apertus judge Zurich German? 50 SwissDial sentences, each as Zurich German (accept), Standard German (reject) and Bernese German (reject); the model answers yes/no (temperature 0).

Judged correctly	70B	8B
All 150 sentences	60 %	58 %

Judged correctly	70B	8B
Zurich German accepted	46 %	44 %
Standard German rejected	72 %	72 %
Bernese German rejected	62 %	58 %

A judge that always says "no" would score 67 %, so both models are below a trivial baseline: they accept less than half of the genuine Zurich sentences and still let 28 % of the Standard German ones pass. In the pilot role-plays (ten learner lines: Standard German, one half-dialect line and natural Zurich German), 70B and 8B each judged correctly in only 5 of 10 cases whether the line was already natural Zurich German; 70B, for example, "corrected" the perfectly idiomatic «Gömmer zäme en Kafi go trinke?» to «Chömmer …». Decision: the app never grades the learner's dialect.

E3 — Zurich German generation and the style guide. 100 SwissDial sentences translated from Standard German to Zurich German (temperature 0), chrF against the SwissDial Zurich transcript, plus two lexical metrics: the share of Zurich forms among words whose form differs between Zurich and Bernese/Basel German (*hät/het, nöd/nid, gsi/gsy, de/dr, und/u …*) and Standard German leftovers per 100 words (*ist, nicht, hat, ein …*). On SwissDial references the share is 100 % for Zurich and 37 % for Bernese, so it separates the dialects; Standard German, however, also scores 100 % (it says *und*), so the share must be read together with the leftovers (Zurich references 0.1, untranslated Standard German 5.9 per 100 words).

Model, prompt	chrF	Zurich share	Standard German leftovers / 100 words
Untranslated input (copy baseline)	50.4	100 %	5.9
70B, baseline prompt	58.6	61 %	0.1
70B, with style guide	58.6	70 %	0.1
8B, baseline prompt	59.6	61 %	0.1
8B, with style guide	58.2	71 %	0.2

The style guide shifts word forms towards Zurich (61 % → 70 % for 70B, 61 % → 71 % for 8B) but does not improve chrF; forms such as *dr*, *het* and *ond* still appear with it. Both models translate rather than copy (chrF about 8 points above the untranslated input), and their outputs are close: 70B and 8B produce the identical sentence for 29 of the 100 inputs.

E4 — Feedback on the learner's answers (63 synthetic items run through the app's `Session`): 19 correct answers, 4 in Swiss Standard German, 20 with one typical error, 20 misunderstandings (correct German that does not fit what was said). In the first design the role-play call also judged the learner's Standard German and comprehension; both models then passed almost every answer as correct and noticed few misunderstandings. We moved both judgements into separate, focused calls (section 2) and measured again; "before" is the first design.

	70B before	70B	8B before	8B
False alarms on correct German	1/35	**0/43**	0/32	**0/40**
… on Swiss Standard German	0/4	0/4	0/2	0/4
Errors detected (with the expected correction)	0/19	**13/20**	0/18	**12/19**
Misunderstandings detected	2/20	**18/20**	1/20	**17/19**
False "misunderstood" alarms	0/43	**0/43**	0/43	**0/42**
No verdict / unparseable reply	9 / 1	0 / 1	13 / 1	4 / 0

With the separate checks both models find about two thirds of the errors without a single false alarm, also on Swiss forms such as *parkieren* or *das Tram*, and 70B notices 18 of 20 misunderstandings without a false alarm on 43 fitting answers; inside the role-play call it had noticed 3 of 20 even after the Standard German check was moved out (8B: 2 of 20). 8B's final run lost four answers to Public AI errors (HTTP 400); the fractions leave them out. Both models miss the

same seven errors: two prepositions (*zu Luzern*, *bei Kunsthaus*), the auxiliary in *habe … gekommen*, the participle *geschwimmt*, a case error (*ein Termin*), a gender error (*einen Halbtax*) and the verb position in a subordinate clause.

Latency (native run on Apple Silicon): Apertus 70B through Hugging Face/Featherless took a median 8.6 s per role-play call (4.6–25.4 s, eight calls; the parallel check is shorter), 8B on Public AI 10.7 s (7.5–21.8 s, ten calls, before the check existed). Speech synthesis with Qwen3-TTS 1.7B on MPS ran at about real time (median 0.99 s per second of audio, lines of 2–6 s); Whisper large-v3-turbo (MLX) transcribed a spoken answer in 0.9 s.

6. Value, cost and scalability

Who it is for. Newcomers who already study Standard German (international staff and students, people in integration programmes) and the organisations that support them: integration offices, language schools, employers. Courses teach Standard German; Los emal! trains the step they leave out, understanding the dialect in the situations where it is spoken, at any time and without a native speaker present.

Cost per use. Only the Apertus calls have a per-use cost; speech recognition and synthesis run in the container. Measured over twelve scripted turns with 70B, a learner turn uses about 3,500 tokens: the role-play call about 1,470 prompt and 230 completion tokens, the Standard German check 260 + 40, the comprehension check 270 + 15, the Zurich version 540 + 10, plus a share of the scene's opening call. An hour of active practice (about 60 turns) therefore needs about 210,000 tokens, or about CHF 0.21 at an illustrative price of CHF 1 per million tokens for a hosted 70B model. Most of it is the fixed instructions of the prompt, which endpoints with prefix caching reuse across turns. The 8B model, which did about as well as 70B in E2–E4, is several times cheaper to serve: in BF16 it fits on one 24 GB GPU, whereas 70B needs about 140 GB of GPU memory.

Scalability. Apart from the per-browser session in memory the app is stateless, so instances can be replicated behind a load balancer; the Apertus endpoint is shared and can be any OpenAI-compatible server (vLLM batches concurrent sessions). Speech is the per-instance cost: on CPU, synthesis takes 3–4.5 s per second of audio and a container needs up to 7.5 GB of memory, so a deployment for many learners should run the speech models on a GPU.

7. Limitations

- Apertus' Zurich German is understandable but not native-like; it mixes in other dialects and Standard German, and the role-play still produces occasional calques and Bernese forms (*dr*). Our style guide reduces but does not remove this. We have not yet had the output rated by native speakers.
- The synthesised voice reads dialect text with a German text-to-speech model; prosody and some vowels are Standard German-like. A Zurich German TTS would need an openly licensed corpus.
- Whisper tends to normalise speech towards fluent Standard German, so some learner errors may disappear before Apertus sees them; typed answers avoid this. It also mishears Swiss terms (*Halbtax* as *Halbtags*), which the Standard German check then "corrects".
- The "how a local would say it" line has the quality measured in E3 and stays marked as experimental. Word explanations sometimes come in German instead of the chosen help language.
- The learner test set is synthetic and small (63 items); real learner data would be needed for a stronger claim.
- On CPU, speech synthesis with the 0.6B model takes 3–4.5 s per second of audio, so a turn in the Docker image takes 20–50 s.

- The shared hosted endpoint is a single point of failure: during our tests Public AI throttled the 70B model (HTTP 429, each held for about 26 s) for about 20 hours while 8B kept answering. The app therefore switches to 8B after the first 429 for the rest of the session (`LLM_FALLBACK_NAME` sets the fallback for other endpoints); other failures are retried for about a minute. A self-hosted vLLM endpoint avoids the shared quota.

8. Reproducibility

- `make run` builds and starts the Docker image (CPU) on a clean checkout; set `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY` (see `.env.example`). The build downloads the Python packages and the speech models (image 6.6 GB, a few minutes). Give Docker at least 8 GB of memory: the container peaked at 7.3 GB in our runs. To fit, the CPU build keeps the TTS text embedding (a third of its weights) in bfloat16 as stored in the checkpoint and decodes speech in 4-second chunks: decoding a 7-second reply in one piece needed 2.4 GB extra, growing with the length, and our first container ran out of memory.
- Native run on Apple Silicon (used for the demo): `uv venv --python 3.11 .venv && uv pip install --python .venv torch torchaudio -r requirements.txt mlx-whisper`, then `ASR_BACKEND=mlx TTS_MODEL=Qwen/Qwen3-TTS-12Hz-1.7B-Base make run-local PY=.venv/bin/python`.
- Evaluations: `make eval-judge eval-generation eval-feedback PY=.venv/bin/python SWISSDIAL=/path/to/swissdial`. E1 runs in its own environment with the Swiss AI transformers fork for Apertus audio input (commit `3797303`) and mlx-whisper: `make venv-asr eval-asr`. Public AI's free tier allows 100 requests per minute; `EVAL_WORKERS` and `EVAL_DELAY` pace the calls.
- `make test` runs the unit tests (no models, no network).
- Hardware used: Apple MacBook Pro (Apple Silicon); Apertus via Public AI.

9. Next steps

- Native-speaker rating of the generated Zurich German and of the style guide's effect.
- An openly licensed Zurich German voice: record consenting speakers and fine-tune Qwen3-TTS.
- Fine-tune Apertus 8B on openly licensed Zurich German dialogue to run fully on a laptop.
- More dialects (Bernese, Basel) with dialect-specific style guides; spaced repetition of the words collected in each scene.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Swiss AI Initiative. *Apertus 1.5* model cards, huggingface.co/swiss-ai.
- P. Dogan-Schönberger, J. Mäder, T. Hofmann. *SwissDial: Parallel Multidialectal Corpus of Spoken Swiss German.* arXiv:2103.11401, 2021.
- A. Radford et al. *Robust Speech Recognition via Large-Scale Weak Supervision* (Whisper), 2022.
- Qwen Team. *Qwen3-TTS* model cards, huggingface.co/Qwen.
- M. Popović. *chrF: character n-gram F-score for automatic MT evaluation.* WMT 2015.